

CSE370 : Database Systems

Project Report

Project Title : The Smart Hostel Ecosystem: A Living Experience

Group No : 10, CSE370 Lab Section : 10, Spring 2026		
ID	Name	Contribution
24101419	Samin Yasar	Researched on the student authentication system, navigation bar, and frontend UI design. Implemented the system of logins, session management, Provided additional student interface such as dashboard, service and visitor request forms, complaints submission, announcement read and meal token collection.
23201547	Md. Reazul Amin	Worked on the warden (admin) module. Adopted complaint management system, service and visitor approval system, task assignment, announcements and meal plan management.
23201211	Walid Mahmud Khan	Worked on the staff module. Introduced task control, meal token checking system, and minimum of token responsibility. Created a backend logic and frontend dashboard of staff operations.

Table of Contents

Section No	Content	Page No
1	Introduction	03
2	Project Features	04
3	ER/EER Diagram	05
4	Schema Diagram	06
5	Normalization	07-08
6	Frontend Development	09-12
7	Backend Development	13-27
8	Source Code Repository	28
9	Conclusion	28
10	References	29

Introduction

This project is a Smart Hostel Management System that automates the operations of the hostels by providing basic access to students, wardens, and staff. It is efficient in its management of room allocation, service requests, complaints and daily activities. The system provides various benefits of coordination, real time tracking, and enhanced living standard in the hostel. The aim of this system is to ensure that students, wardens and staff members have a smart, efficient and user-friendly environment with role-based access and custom dashboards. All users, be it a student, warden or a staff member, interact with the system depending on their responsibilities, which ensures a better organization and security.

The site contains the important details of the hostel including room assignment and room availability.

service and maintenance requests, complaint tracking and resolution, Meal planning and token system. Another one of the eye-catching features is the service request system, where the students are able to report the problems, such as electrical, plumbing problems or any other problems. Moreover, the system refreshes daily convenience in terms of keeping features such as status tracking in real-time.

In general, this project will be focused on changing the ordinary hostel management into a smart, well maintained and easy to manage system; enhancing efficiency, transparency and overall living experience to the students as well as reducing manual workload to the administrators.

Project Features

ID, Name	Features [3 per member]	
20101999, Samin Yasar	Ft 1	Student authentication and session management system.
	Ft 2	Student navigation bar and UI interface.
	Ft 3	Student interaction features (complaint, service, visitor forms UI).
23201547, Md.Reazul Amin	Ft 1	Complaint management system for wardens.
	Ft 2	Visitor and service request approval system.
	Ft 3	Task,meal assignment and announcement system.
Walid Mahmud Khan, DEF	Ft 1	Staff task management system.
	Ft 2	Meal token verification system.
	Ft 3	Token duty assignment and operational management.

Normalization

- a. Explain if your converted Schema is in 1NF or not. If not, decompose it to 1NF.
- b. Explain if your converted Schema is in 2NF or not. If not, decompose it to 2NF. Can there be any partial functional dependencies in your relational schema?
- c. Explain if your converted Schema is in 3NF or not. If not, decompose it to 3NF. Can there be any transitive dependencies in your relational schema?

a. The schema is in 1NF, as there are no multivalued or composite attributes, also no nested tables or values.

b. The schema is in 2NF, as it's already in 1NF, and most tables are single primary tables, there is no partial functional dependency on primary key, also there are no non-key attributes in the composite tables.

c. But the schema is not in 3NF, there are some transitive dependencies in the Student, Warden, Staff tables. In 'Student' table,
 $user_id \rightarrow phone, password, name, email, room_no, plan_id, token_id$; but
 $room_no \rightarrow capacity, floor, type$
 $plan_id \rightarrow menu, date, type$
 $token_id \rightarrow timestamp, meal_type, status$

Also, in Staff table,

$user_id \rightarrow s_id$

$s_id \rightarrow type, problem_desc, status$

These cause transitive dependencies.

To solve this problem, we do 3NF Normalization.

Final 3NF schema:

Student_info: $user_id, phone, password, name, email$

Student: $user_id, room_no, plan_id$

Student_token: $user_id, token_id$

Warden: user_id, phone, password, name, email

Staff: user_id, phone, password, name, s_id

Room: room_no, capacity, floor, type

ServiceReq: s_id, type, problem_desc, status, user_id

VisitRequest: req_id, visit, name, status, user_id

Docs: doc_id, req_id

Complaint: complaint_id, description, admin_rv, status, user_id

Announcement: announcement_id, m_text, date_post, wuser_id

Reads: user_id, announcement_id

MealPlan: plan_id, menu, date, type, wuser_id

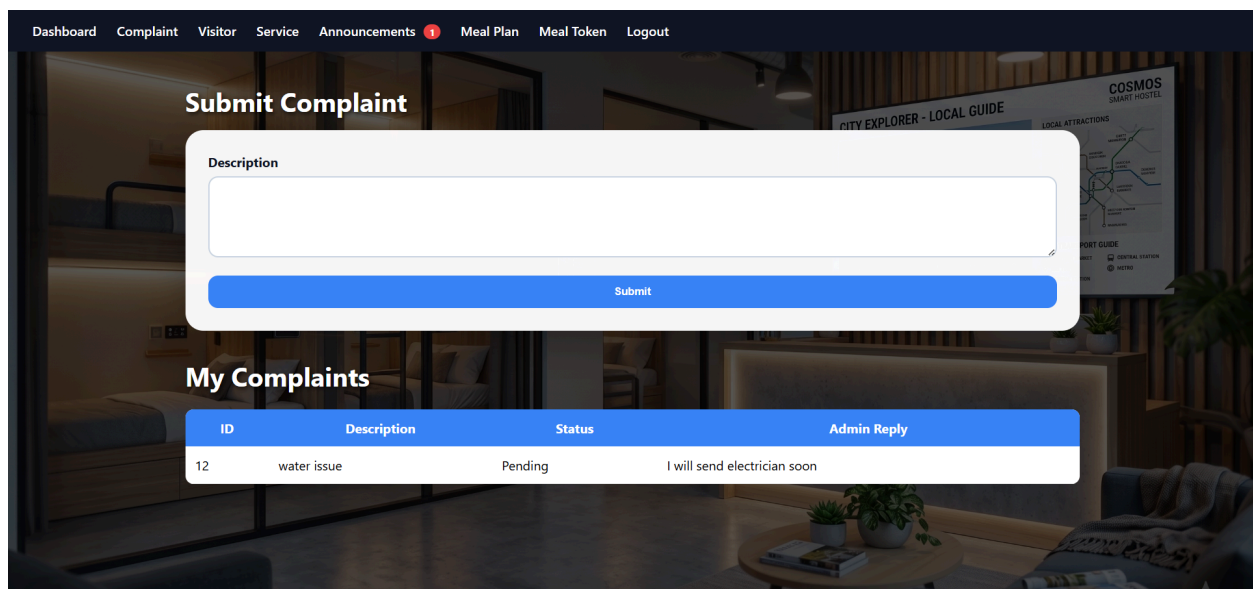
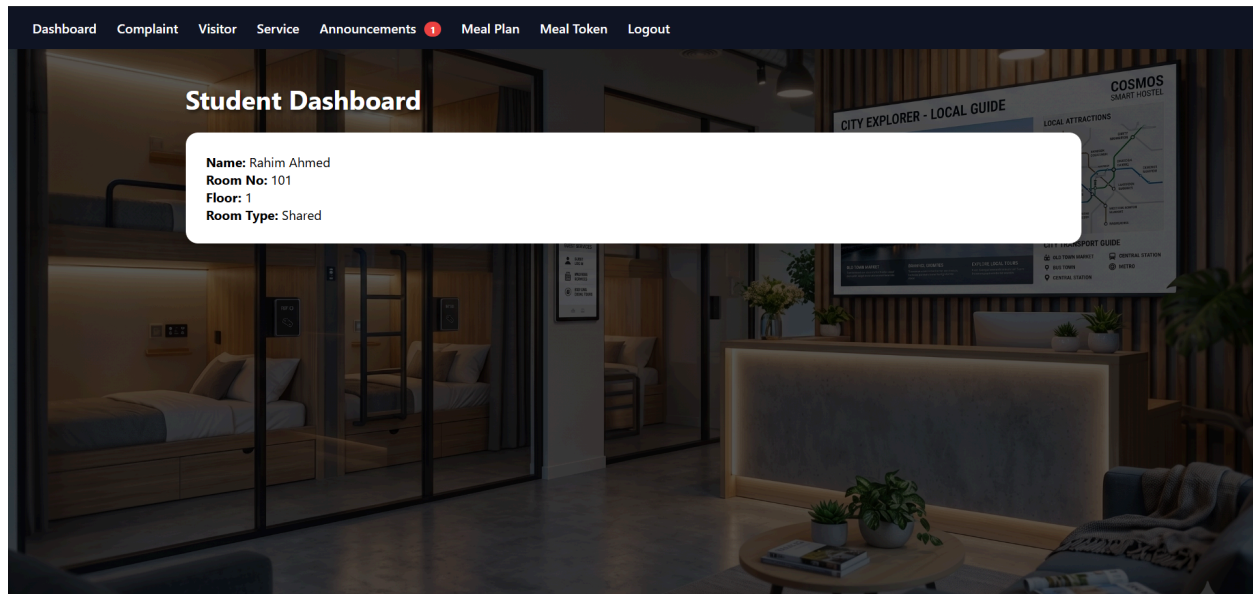
MealToken: token_id, timestamp, meal_type, status, plan_id

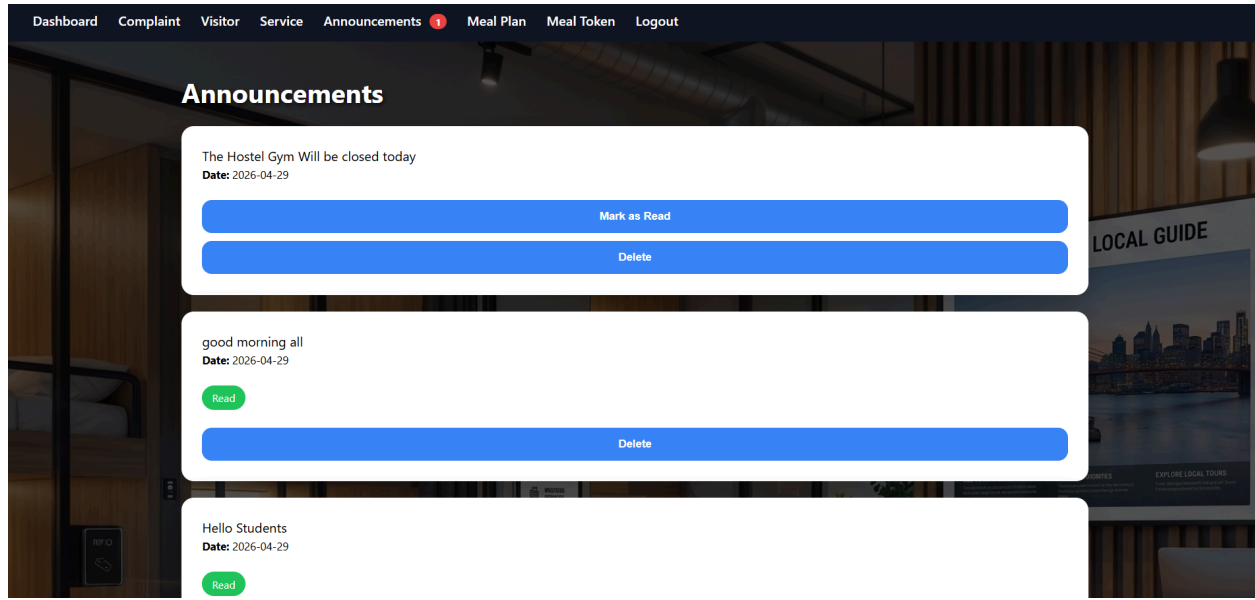
As all transitive dependencies are removed, thus satisfying the 3NF.

Frontend Development

Contribution of ID : 20101999, Name : Samin Yasar

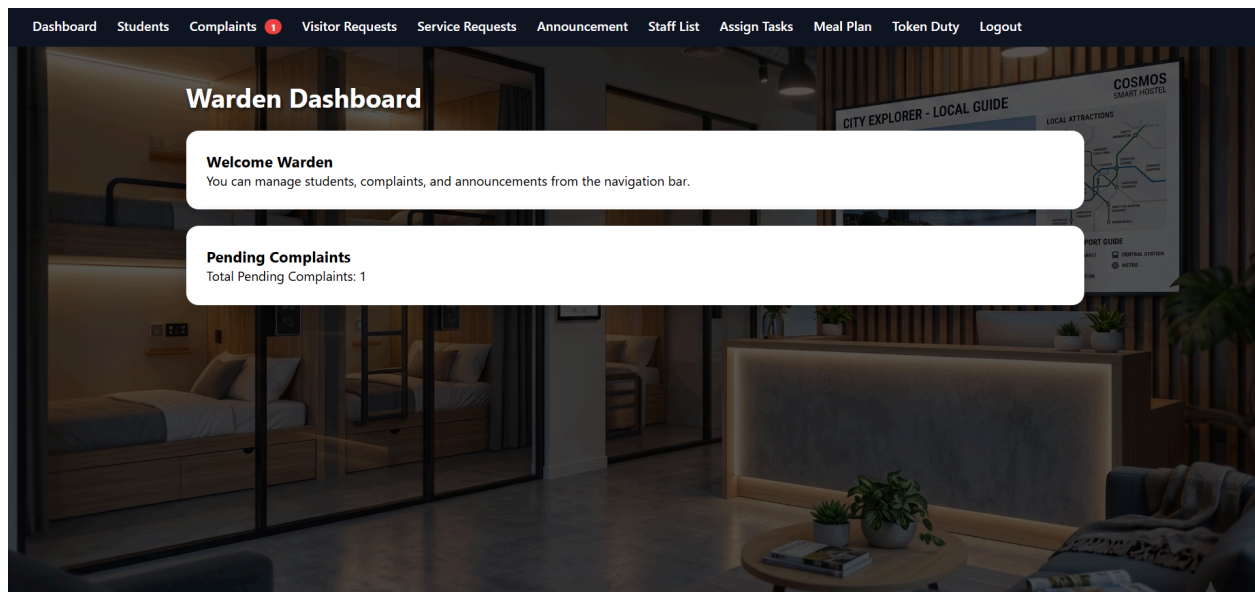
Created the student frontend interface with the following elements; Navigation bar, login page, and dashboard UI. Created clean and responsive Web designs with CSS and added the features of JavaScript to enhance user interaction. Created navigation bar for students to access features like complaints, service requests, visitor requests, announcements, and meal system. Forms to be filled in by the user and correct alignment of UI elements.

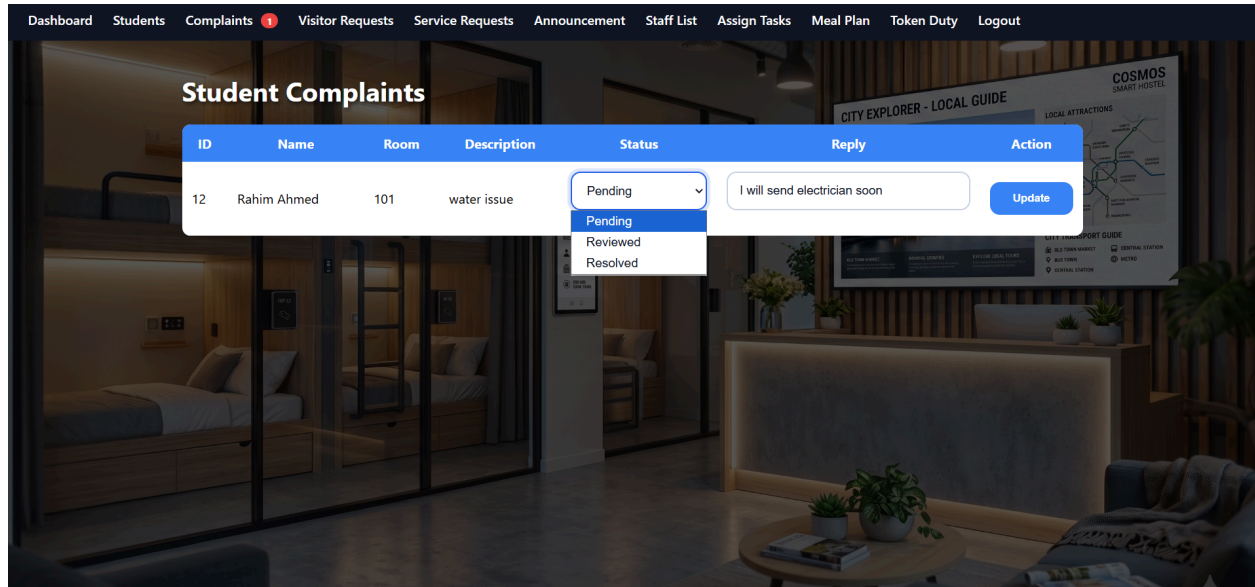




Contribution of ID : 20101998, Name : Md.Reazul Amin

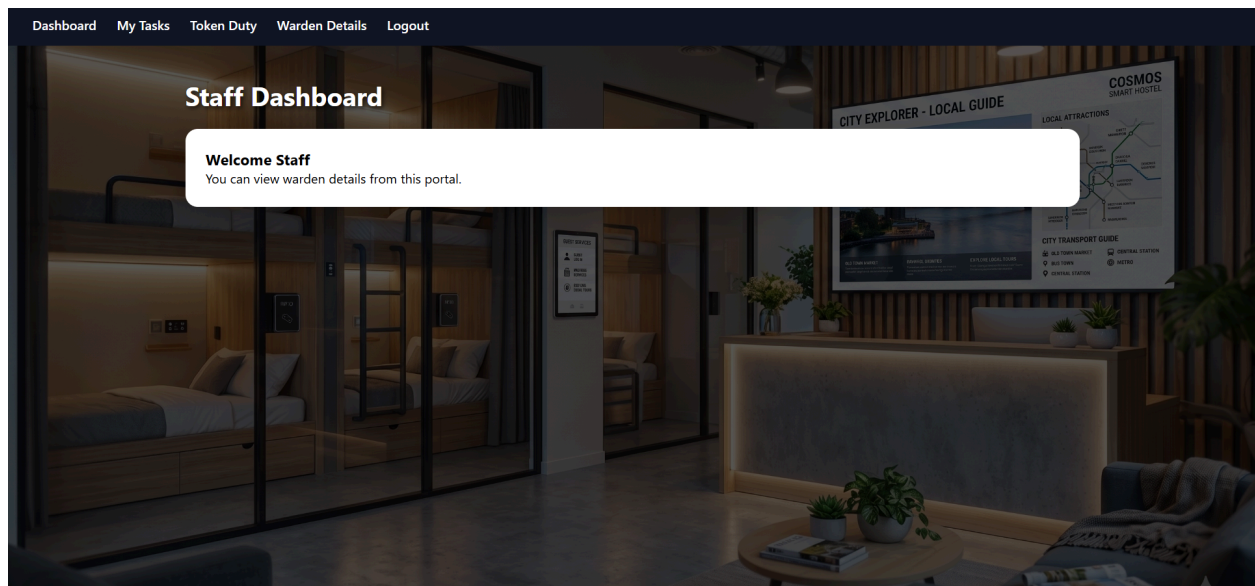
Created frontend interfaces of the warden dashboard. Form and table designed to handle complaints, granting of services, granting of visitor permission, and delegation of tasks. Designed interactive UI features like dropdown menu to update status and text field to respond. Established uniform design of all the pages of the administration.





Contribution of ID : 20101997, Name : Walid Mahmud Khan

Created frontend interfaces to staff dashboard. Specialized tables of managing tasks and tokens. Introduced buttons and forms to indicate the work done and the meal tokens operations. Kept UI coherence with the system design as a whole.



DashboardMy TasksToken DutyWarden DetailsLogout

My Tasks

ID	Task	Description	Date	Time	Assigned By	Status	Action
27	Cleaning room	Sweep the rooms.	2026-05-05	18:26:00	Warden Sir	Done	Completed
24	clean Room	b	2026-04-29	11:22:00	Warden Sir	Done	Completed
22	clean Room	go to N-519	2026-04-29	09:37:00	Warden Sir	Done	Completed

DashboardMy TasksToken DutyWarden DetailsLogout

My Token Duties

Breakfast
Date: 2026-04-30
Status: Assigned

Evening Snacks
Date: 2026-04-29
Status: Assigned

Evening Snacks
Date: 2026-04-29
Status: Assigned

Lunch
Date: 2026-04-29
Status: Assigned

Student Tokens

Token ID	Student	Room	Meal	Date	Status	Action
----------	---------	------	------	------	--------	--------

Backend Development

Contribution of ID : 24101419 , Name : Samin Yasar

The student module backend implementation is geared towards user authentication, session management, and data entry of student activities.

A connection to a centralized database is made via the db.php file using MySQLi which allows all backend scripts to interact with the MySQL database. This file is present on every student related page to run SQL queries.

The authentication system is enforced in the login.php, where the user credentials (user_id, password and role) are compared with the corresponding database table (Student_Info). When the user successfully logs in, the session variables like the user id of the user, user role are initialized. These are session variables that are used in access control throughout the system.

The auth_student.php file verifies the existence of a valid session, and whether the user who is logged in has the appropriate role. If authentication fails, the user is redirected to the login page. This guarantees a secure access control and eliminates unauthorized usage.

There are several backend operations that are also managed by the student module:

Complaint submission: Student complaints are entered into the Complaint table and start out with a Pending initial status.

Service request entry: Service requests are entered into the table of ServiceReq with its type, description and status.

Entry of visitor request: Visitor information is captured into the visitrequest table with date and time columns.

Collection of meal tokens: The system will first look at whether there is an existing token with the same user, date and type of meal. This avoids production of duplicated tokens.

The implementation of these operations is done through SQL INSERT queries and SELECT queries which are used to fetch and display user specific data like complaints, requests and tokens.

In general, this module guarantees a safe system of logins, appropriate work with sessions, and correct storage of information about student activities.

```

1  <?php
2  if (session_status() === PHP_SESSION_NONE) {
3      session_start();
4  }
5
6  if (!isset($_SESSION['user_id']) || !isset($_SESSION['role']) || $_SESSION['role'] !== "student") {
7      header("Location: /CSE370_Project/login.php");
8      exit();
9  }
10 ?>

```

```

<?php
include("../includes/auth_student.php");
include("../db.php");

$user_id = $_SESSION['user_id'];
$msg = "";

// SUBMIT
if (isset($_POST['submit'])) {
    $desc = $conn->real_escape_string($_POST['description']);

    $conn->query("
        INSERT INTO Complaint (description, status, user_id)
        VALUES ('$desc', 'Pending', '$user_id')
    ");

    header("Location: complaint.php?success=1");
    exit();
}

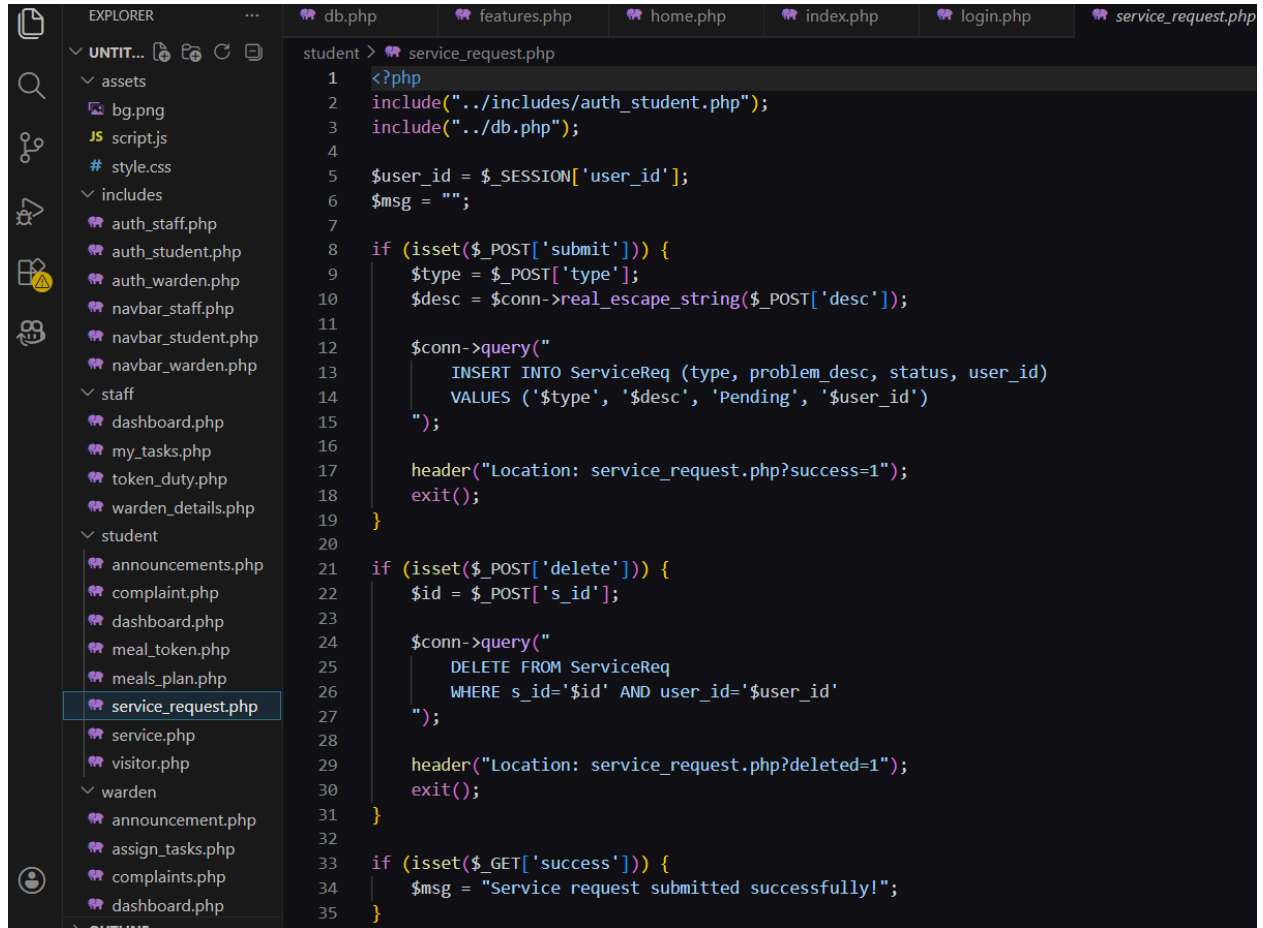
// FETCH
$result = $conn->query("
    SELECT * FROM Complaint
    WHERE user_id='$user_id'
    ORDER BY complaint_id DESC
");

if (isset($_GET['success'])) {
    $msg = "Complaint submitted successfully!";
}
?>

```

```

1  <?php
2  include("../includes/auth_student.php");
3  include("../db.php");
4
5  $user_id = $_SESSION['user_id'];
6  $msg = "";
7  $today = date("Y-m-d");
8
9  if (isset($_POST['collect_token'])) {
10     $meal_type = $_POST['meal_type'];
11
12     $check = $conn->prepare("
13         SELECT token_id FROM DailyMealToken
14         WHERE user_id=? AND meal_type=? AND token_date=?
15     ");
16     $check->bind_param("iss", $user_id, $meal_type, $today);
17     $check->execute();
18     $checkResult = $check->get_result();
19
20     if ($checkResult->num_rows > 0) {
21         header("Location: meal_token.php?msg=already");
22         exit();
23     } else {
24         $stmt = $conn->prepare("
25             INSERT INTO DailyMealToken (user_id, meal_type, token_date, status)
26             VALUES (?, ?, ?, 'Collected')
27         ");
28         $stmt->bind_param("iss", $user_id, $meal_type, $today);
29         $stmt->execute();
30
31         header("Location: meal_token.php?msg=collected");
32         exit();
33     }
34 }
  
```



```
1 <?php
2 include("../includes/auth_student.php");
3 include("../db.php");
4
5 $user_id = $_SESSION['user_id'];
6 $msg = "";
7
8 if (isset($_POST['submit'])) {
9     $type = $_POST['type'];
10    $desc = $conn->real_escape_string($_POST['desc']);
11
12    $conn->query("
13        INSERT INTO ServiceReq (type, problem_desc, status, user_id)
14        VALUES ('$type', '$desc', 'Pending', '$user_id')
15    ");
16
17    header("Location: service_request.php?success=1");
18    exit();
19 }
20
21 if (isset($_POST['delete'])) {
22     $id = $_POST['s_id'];
23
24     $conn->query("
25         DELETE FROM ServiceReq
26         WHERE s_id='$id' AND user_id='$user_id'
27     ");
28
29     header("Location: service_request.php?deleted=1");
30     exit();
31 }
32
33 if (isset($_GET['success'])) {
34     $msg = "Service request submitted successfully!";
35 }
```

```

<?php
include("../includes/auth_student.php");
include("../db.php");

$user_id = $_SESSION['user_id'];

if (isset($_POST['mark_read'])) {
    $announcement_id = $_POST['announcement_id'];

    $conn->query("
        INSERT IGNORE INTO Announcement_Reads (user_id, announcement_id)
        VALUES ('$user_id', '$announcement_id')
    ");
}

if (isset($_POST['delete_announcement'])) {
    $announcement_id = $_POST['announcement_id'];

    $conn->query("DELETE FROM Announcement_Reads WHERE announcement_id='$announcement_id'");
    $conn->query("DELETE FROM Announcement WHERE announcement_id='$announcement_id'");
}

$query = "
SELECT
    a.announcement_id,
    a.m_text,
    a.date_post,
    ar.user_id AS read_status
FROM Announcement a
LEFT JOIN Announcement_Reads ar
    ON a.announcement_id = ar.announcement_id
    AND ar.user_id = '$user_id'
ORDER BY a.announcement_id DESC
";

$result = $conn->query($query);
?>

```

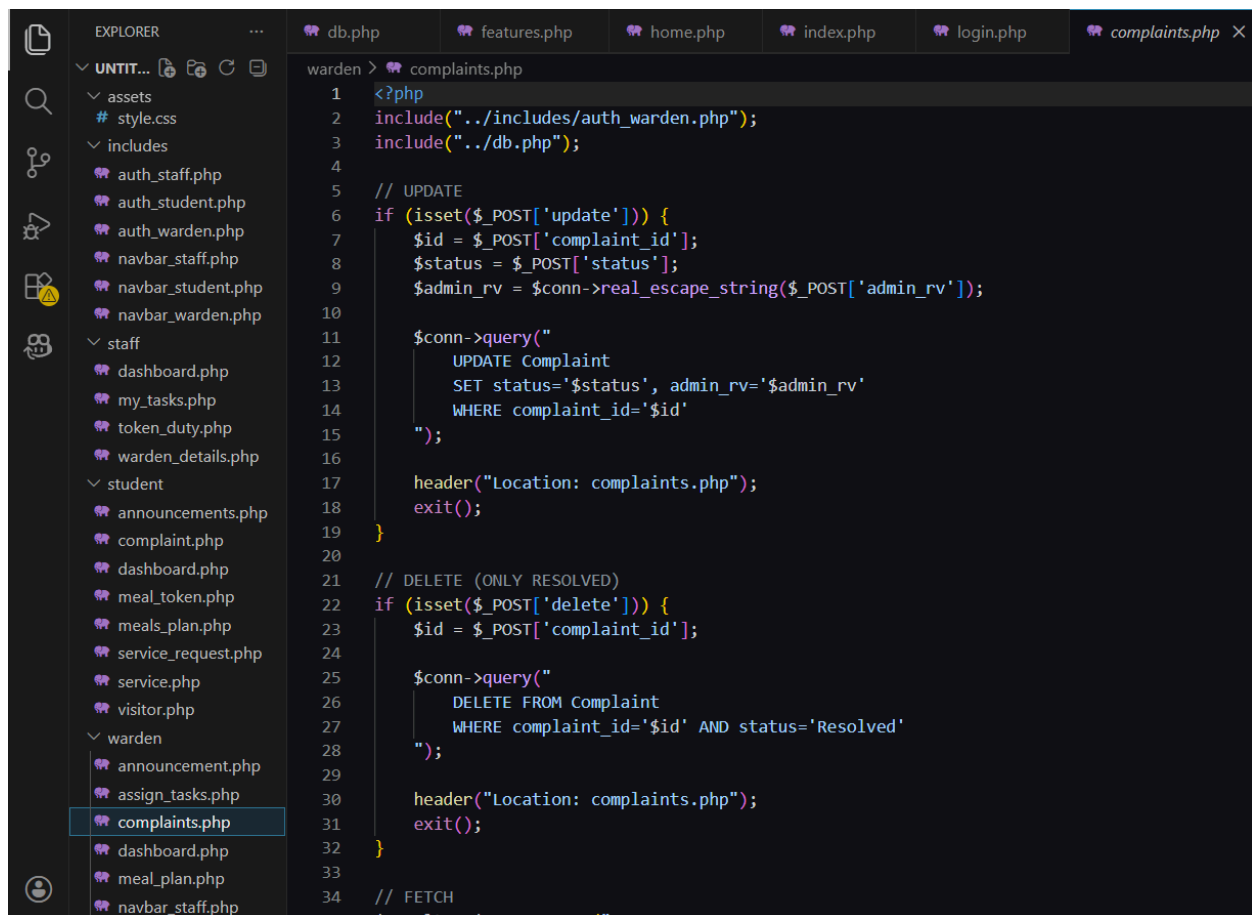
Contribution of ID : 23201547, Name : Md.Reazul Amin

The warden module is the key control system of the Smart Hostel Ecosystem. The back-end logic that will be used in this module will deal with data retrieval, updates, approvals, and task management.

This module includes several SQL queries to carry out CRUD operations on various tables including Complaint, ServiceReq, visitrequest, Announcement, StaffTask, and WeeklyMealPlan.

1. Complaint Management:

This system uses SELECT queries to retrieve complaints, and presents the complaints in a structured form. The warden will be able to record the status of complaints (Pending, Reviewed, Resolved) and respond to complaints on an administrative basis. These updates are implemented with the help of SQL UPDATE statements. Moreover, resolved complaints may be deleted with the help of DELETE statements.



```

1 <?php
2 include("../includes/auth_warden.php");
3 include("../db.php");
4
5 // UPDATE
6 if (isset($_POST['update'])) {
7     $id = $_POST['complaint_id'];
8     $status = $_POST['status'];
9     $admin_rv = $conn->real_escape_string($_POST['admin_rv']);
10
11     $conn->query("
12         UPDATE Complaint
13         SET status='$status', admin_rv='$admin_rv'
14         WHERE complaint_id='$id'
15     ");
16
17     header("Location: complaints.php");
18     exit();
19 }
20
21 // DELETE (ONLY RESOLVED)
22 if (isset($_POST['delete'])) {
23     $id = $_POST['complaint_id'];
24
25     $conn->query("
26         DELETE FROM Complaint
27         WHERE complaint_id='$id' AND status='Resolved'
28     ");
29
30     header("Location: complaints.php");
31     exit();
32 }
33
34 // FETCH
35 $result = $conn->query("

```

2. Service and Visitor Request Handling:

Student requests that are submitted as service and visitor requests are fetched by use of SELECT queries. The warden is allowed to approve or reject such requests, which will update the status

field in the corresponding tables. This will enable good monitoring and approval of hostel activities.

```
<?php
include("../includes/auth_warden.php");
include("../db.php");

// Update status
if (isset($_POST['update'])) {
    $id = $_POST['s_id']; // Changed to match Student table
    $status = $_POST['status'];

    $conn->query("UPDATE ServiceReq SET status='$status' WHERE s_id='$id'"); // Changed table and ID name
}

// Delete request
if (isset($_POST['delete'])) {
    $id = $_POST['s_id']; // Changed to match Student table
    $conn->query("DELETE FROM ServiceReq WHERE s_id='$id'"); // Changed table and ID name
}

// Fetch data
$result = $conn->query("
    SELECT sr.*, si.name
    FROM ServiceReq sr
    JOIN Student_Info si ON sr.user_id = si.user_id
    ORDER BY sr.s_id DESC
");
?>
```

```

1  <?php
2  include("../includes/auth_warden.php");
3  include("../db.php");
4
5  $msg = "";
6
7  // UPDATE
8  if (isset($_POST['update'])) {
9      $id = $_POST['req_id'];
10     $status = $_POST['status'];
11
12     $conn->query("
13         UPDATE visitrequest
14         SET status='$status'
15         WHERE req_id='$id'
16     ");
17
18     header("Location: visitor_requests.php?updated=1");
19     exit();
20 }
21
22 // DELETE
23 if (isset($_POST['delete'])) {
24     $id = $_POST['req_id'];
25
26     $conn->query("
27         DELETE FROM visitrequest
28         WHERE req_id='$id'
29     ");
30
31     header("Location: visitor_requests.php?deleted=1");
32     exit();
33 }
34
35 if (isset($_GET['updated'])) $msg = "Updated successfully!";
36 if (isset($_GET['deleted'])) $msg = "Deleted successfully!";

```

3. Announcement System:

The wardens can generate announcements that are added in the Announcement table as a result of INSERT queries. Every announcement contains the message, date and ID of the warden who posted it. This data is later written back by using SELECT queries and read-tracking is managed

by an extra table.

```

warden > announcement.php
1  <?php
2
3  include("../includes/auth_warden.php");
4  include("../db.php");
5
6  if (!isset($_SESSION['user_id']) || $_SESSION['role'] != "warden") {
7      header("Location: ../login.php");
8      exit();
9  }
10
11 $res = $conn->query("SELECT COUNT(*) AS total FROM Complaint WHERE status='Pending'");
12 $row = $res->fetch_assoc();
13 $comp_count = $row['total'];
14
15 $msg = "";
16
17 if ($_SERVER["REQUEST_METHOD"] == "POST") {
18     $text = $_POST['text'];
19     $user = $_SESSION['user_id'];
20
21     if (!empty($text)) {
22         $conn->query("INSERT INTO Announcement (m_text, date_post, wuser_id)
23                     VALUES ('$text', CURDATE(), '$user')");
24         $msg = "Posted!";
25     }
26 }
27 ?>

```

4. Task Assignment System:

The warden assigns tasks to staff members by inserting records into the StaffTask table. Each task contains details such as title, description, date, time, assigned staff, and status. This

establishes a connection between the warden and staff modules.

```

warden > assign_tasks.php
1  <?php
2  include("../includes/auth_warden.php");
3  include("../db.php");
4
5  $warden_id = $_SESSION['user_id'];
6  $msg = "";
7
8  if (isset($_POST['delete_task'])) {
9      $task_id = $_POST['task_id'];
10
11      $conn->query("DELETE FROM StaffTask WHERE task_id='$task_id'");
12      $msg = "Completed task deleted successfully!";
13  }
14
15  $comp_count = 0;
16  $countQuery = $conn->query("SELECT COUNT(*) AS total FROM Complaint WHERE status='Pending'");
17  if ($countQuery) {
18      $countRow = $countQuery->fetch_assoc();
19      $comp_count = $countRow['total'];
20  }
21
22  if ($_SERVER["REQUEST_METHOD"] == "POST") {
23      $task_title = $_POST['task_title'];
24      $task_desc = $_POST['task_desc'];
25      $task_date = $_POST['task_date'];
26      $task_time = $_POST['task_time'];
27      $staff_id = $_POST['staff_id'];
28
29      $query = "INSERT INTO StaffTask
30      (task_title, task_desc, task_date, task_time, status, warden_id, staff_id)
31      VALUES
32      ('$task_title', '$task_desc', '$task_date', '$task_time', 'Assigned', '$warden_id', '$staff_id')";
33
34      if ($conn->query($query)) {
35          header("Location: assign_tasks.php?success=1");
36          exit();
37      }

```

5. Meal Plan Management:

The weekly meal plan is managed using the WeeklyMealPlan table. The warden updates meal items for specific days and meal types using UPDATE queries. Students can later view this plan through SELECT queries. The warden selects on what days what should be given as meal and then it

```

warden > meal_plan.php
1  <?php
2  include("../includes/auth_warden.php");
3  include("../db.php");
4
5  mysqli_report(MYSQLI_REPORT_OFF);
6
7  $warden_id = $_SESSION['user_id'];
8  $msg = "";
9  $error = "";
10
11 $res = $conn->query("SELECT COUNT(*) AS total FROM Complaint WHERE status='Pending'");
12 $row = $res ? $res->fetch_assoc() : ["total" => 0];
13 $comp_count = $row['total'];
14
15 $days = ["Saturday", "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday"];
16
17 $mealColumns = [
18     "breakfast" => "breakfast",
19     "morning_snacks" => "morning_snacks",
20     "lunch" => "lunch",
21     "evening_snacks" => "evening_snacks",
22     "dinner" => "dinner"
23 ];
24
25 if (isset($_POST['save_meal'])) {
26     $day = $_POST['day_name'];
27     $meal_type = $_POST['meal_type'];
28     $items = $_POST['items'];
29
30     if (!array_key_exists($meal_type, $mealColumns)) {
31         $error = "Invalid meal type selected.";
32     } else {
33         $column = $mealColumns[$meal_type];
34
35         $checkStmt = $conn->prepare("SELECT plan_id FROM WeeklyMealPlan WHERE day_name = ?");
36         $checkStmt->bind_param("s", $day);
37         $checkStmt->execute();

```

6. Token Duty Assignment:

The system assigns staff members to meal duties by inserting records into the TokenDuty table. This ensures that meal token verification is handled by designated staff members.

```

warden > token_duty.php
1  <?php
2  include("../includes/auth_warden.php");
3  include("../db.php");
4
5  $warden_id = $_SESSION['user_id'];
6  $msg = "";
7
8  $res = $conn->query("SELECT COUNT(*) AS total FROM Complaint WHERE status='Pending'");
9  $row = $res->fetch_assoc();
10 $comp_count = $row['total'];
11
12 if (isset($_POST['assign_duty'])) {
13     $staff_id = $_POST['staff_id'];
14     $duty_date = $_POST['duty_date'];
15     $meal_type = $_POST['meal_type'];
16
17     $stmt = $conn->prepare("
18         INSERT INTO TokenDuty (staff_id, duty_date, meal_type, assigned_by, status)
19         VALUES (?, ?, ?, ?, 'Assigned')
20     ");
21     $stmt->bind_param("issii", $staff_id, $duty_date, $meal_type, $warden_id);
22     $stmt->execute();
23
24     header("Location: token_duty.php?success=assigned");
25     exit();
26 }
27
28 if (isset($_GET['success'])) {
29     $msg = "Token duty assigned successfully!";
30 }
31
32 $staffs = $conn->query("SELECT user_id, name FROM Staff ORDER BY name ASC");
33
34 $duties = $conn->query("
35     SELECT td.*, s.name AS staff_name
36     FROM TokenDuty td
37     JOIN Staff s ON td.staff_id = s.user_id

```

Overall, the backend of the warden module ensures centralized control, efficient decision-making, and smooth communication between students and staff.

Contribution of ID : 23201211, Name : Walid Mahmud Khan

1. Task Management

Assigned tasks are retrieved from the StaffTask table using SELECT queries, along with warden details through SQL JOIN operations. Tasks are displayed in descending order based on task ID.

When a staff member completes a task, the system updates the task status to Done using an UPDATE query. This ensures real-time tracking of task completion.

2. Meal Token Verification

The meal token system is implemented using the DailyMealToken table. Staff members verify tokens collected by students. When a token is used, its status is updated from Collected to Used, and additional fields such as used_at and used_by_staff_id are recorded.

Prepared statements are used in this part to securely update token data, which improves protection against SQL injection.

3. Token Duty Management

Staff members view their assigned duties from the TokenDuty table using SELECT queries. Only tokens corresponding to assigned duty dates and meal types are displayed. This is ensured using SQL subqueries with EXISTS.

4. Token Deletion

Used tokens can be deleted by the same staff member who processed them. This operation is performed using a DELETE query with conditions to ensure data integrity.

5. Warden Information Retrieval

Staff members can view warden details, which are fetched from the Warden table using SELECT queries. This helps maintain coordination between staff and administrators.

6. Notification System

The system calculates pending tasks and token duties using aggregate SQL functions such as COUNT(). These values are displayed as notification badges in the staff navigation bar.

Overall, the staff module backend ensures efficient task execution, controlled token management, and proper coordination with the warden module.

```
staff > my_tasks.php
1  <?php
2  include("../includes/auth_staff.php");
3  include("../db.php");
4
5  $staff_id = $_SESSION['user_id'];
6  $msg = "";
7
8  if (isset($_POST['done'])) {
9      $task_id = $_POST['task_id'];
10
11     $conn->query("UPDATE StaffTask SET status='Done' WHERE task_id='$task_id' AND staff_id='$staff_id'");
12     $msg = "Task marked as done!";
13 }
14
15 $query = "
16 SELECT
17     t.task_id,
18     t.task_title,
19     t.task_desc,
20     t.task_date,
21     t.task_time,
22     t.status,
23     w.name AS warden_name
24 FROM StaffTask t
25 JOIN Warden w ON t.warden_id = w.user_id
26 WHERE t.staff_id = '$staff_id'
27 ORDER BY t.task_id DESC
28 ";
29
30 $result = $conn->query($query);
31 ?>
```

```

staff > token_duty.php
1  <?php
2  include("../includes/auth_staff.php");
3  include("../db.php");
4
5  $staff_id = $_SESSION['user_id'];
6  $msg = "";
7
8  if (isset($_POST['cut_token'])) {
9      $token_id = $_POST['token_id'];
10
11      $stmt = $conn->prepare("
12          UPDATE DailyMealToken
13          SET status='Used', used_at=NOW(), used_by_staff_id=?
14          WHERE token_id=? AND status='collected'
15      ");
16      $stmt->bind_param("ii", $staff_id, $token_id);
17      $stmt->execute();
18
19      header("Location: token_duty.php?success=cut");
20      exit();
21  }
22
23  if (isset($_POST['delete_token'])) {
24      $token_id = $_POST['token_id'];
25
26      $conn->query("
27          DELETE FROM DailyMealToken
28          WHERE token_id='$token_id'
29          AND status='Used'
30          AND used_by_staff_id='$staff_id'
31      ");
32
33      header("Location: token_duty.php?success=deleted");
34      exit();
35  }
36
37  if (isset($_GET['success'])) {

```

Ln 1, Col 1 Spaces: 4 UTF-8 CRI

Source Code Repository

<https://github.com/Samin-Yasar-CSE/CSE370-Project>

Conclusion

The Smart Hostel Ecosystem is a well organized and efficient platform to manage hostel operations. It combines student interaction, administrative control and staff operations into one system successfully.

The system improves communication, reduces manual effort and increases overall efficiency. Potential future improvements include better security, mobile responsiveness and advanced analytics.

References

1. SuperSimpleDev. (n.d.). HTML & CSS full course
<https://supersimpledev.github.io/references/html-css-reference.pdf>
2. Bro Code. (n.d.). PHP full course for free [Video]. YouTube.
<https://youtu.be/zZ6vybT1HQs?si=Ge0hen-bhGgFoP3r>
3. W3Schools. (n.d.). PHP time() function.
https://www.w3schools.com/php/func_date_time.asp
4. W3Schools. (n.d.). HTML tables.
https://www.w3schools.com/html/html_tables.asp